

現行 AIPL による N-クイーンズの実機ベンチマーク

—— Xinu / Raspberry Pi 5 上での実測

児玉

2026 年 8 月 30 日

要旨 現行仕様 (2026-07-31 版) の表層構文で書いた N-クイーンズを、自作 OS Xinu が走る Raspberry Pi 5 実機の AIPL 実行系に載せて測った。N = 4-7 で解の個数は**すべて厳密解と一致**した。一方で所要時間の大半は探索ではなく **HTTP 往復と JIT コンパイル (中央値 91.8 ms)** が占め、探索そのものが雑音を超えて見えるのは $N \geq 6$ からである。N ≥ 8 は**完了しない** (アクタが 848 体残ったまま停止する)。アクタ 1 体あたりの費用は N = 6 で 80 μs、N = 7 で 108 μs と同じ桁に収まった。

1 測ったもの

1.1 対象

実行基盤	Raspberry Pi 5 (BCM2712 / Cortex-A76、4 コア稼働) 上の自作 Xinu。 カーネル kernel_2712.img、Xinu Round 1 / BCM2712、EL1 動作
AIPL 実行系	Mac 側 aipl2c --xinu-jit が C を生成し、POST /actor/load でボードへ送る。ボード上で JIT コンパイルして実行する
言語	現行 AIPL の表層構文 。戻り値注釈 : T、効果注釈 !{mut}、真偽値リテラル true/false、文字列連結 ++ を使用
測定点	Mac から見た POST /actor/load の総所要時間 (curl の time_total)

1.2 差し引きの設計

測定点はネットワーク往復と JIT コンパイルを含んでしまう。そこで各 N について**同一の C コード**を二通りの引数で走らせ、差を取った。

- **本番** : bootstrap(N, 0, N) —— 0 行目の列を全部担当する (=探索する)
- **素通し** : bootstrap(N, 0, 0) —— 担当区間が空なので**探索しない**

素通しは本番と**バイト数まで同一** (5660 B) なので、ネットワーク往復と JIT コンパイルの費用は同じだけかかる。両者の差が探索そのものの費用になる。各 N につき 5 回ずつ、本番と素通しを交互に、間にアクタ GC を挟んで測った (全 20 組)。

2 結果

2.1 解の正しさ

N	得られた解の個数	厳密解	探索木の節点数	判定
4	2	2	17	一致 (5/5 回)
5	10	10	54	一致 (5/5 回)
6	4	4	153	一致 (5/5 回)
7	40	40	552	一致 (5/5 回)

20 回すべてで厳密解と一致した。節点数は同じ枝刈りアルゴリズムを Mac 側で数えたもので、測定値ではなくモデルである。

2.2 所要時間

N	本番 最小	本番 中央値	本番 最大	探索 (中央値 - 基準)	アクタ 1 体あたり
4	80.8 ms	92.0 ms	258.7 ms	+0.3 ms	— (雑音以下)
5	83.5 ms	87.2 ms	97.8 ms	-4.6 ms	— (雑音以下)
6	99.9 ms	104.0 ms	108.3 ms	+12.2 ms	80.3 μ s
7	144.1 ms	151.2 ms	154.3 ms	+59.4 ms	107.8 μ s

基準（素通し）は全 20 試行を込みにした中央値 **91.8 ms**（最小 76.4 ms / 最大 100.7 ms）。雑音の幅は **24.2 ms** あり、 $N = 4, 5$ の差はこの中に埋もれている。 $N = 5$ の -4.6 ms という負の値は、探索が速いのではなく測っていないのと同じということである。

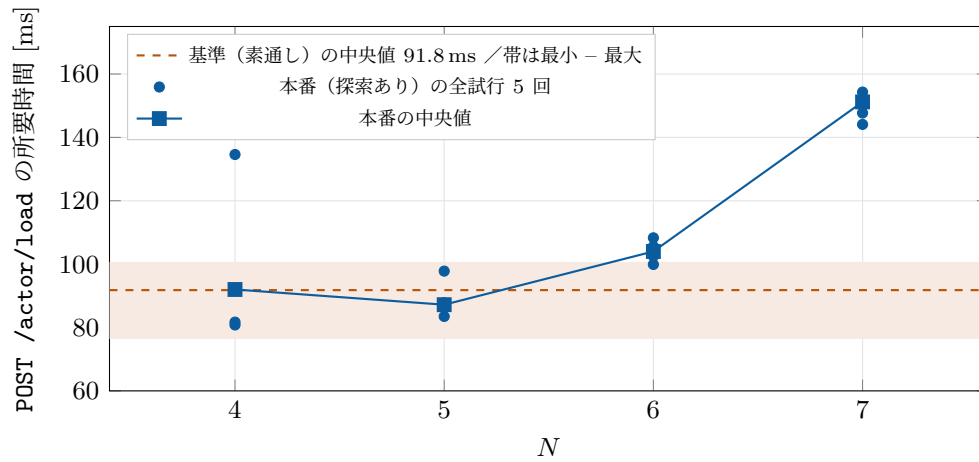


図 1: 全試行をそのまま描いた図。 $N = 4, 5$ の点は基準の帯（橙）の中に収まっており、探索の費用は測定雑音より小さい。 $N = 6$ で帯の上に出て、 $N = 7$ で明確に離れる。 $N = 4$ の 258.7 ms は 1 回だけの外れ値で図の範囲外にあり、中央値には効いていない。

2.3 $N \geq 8$ は完了しない

$N = 8$ を投入すると [NQ] ...solutions= の出力が出ず、応答は actors: 848 live で終わる。そのあと /actor/send?m=get_done を 6 回突ついても done は立たず、解の個数も 0 のままだった。すなわち

探索木が排出しきれずに止まっている。 $N = 9$ 、 $N = 10$ でも同様に、残存アクタは1014体、1023体だった（アクタ表の上限が1024であることを示す）。

これは今回の言語追従とは無関係で、**実行系側の限界**である。なお同じ限界は Pi 4 でも記録されている（ $N = 8$ で未完了）。

3 読み取れること

- 現行構文の AIPL が実機で動く。: T /!{mut} / true / ++ を使ったプログラムが、Mac で C に落ち、ボード上で JIT コンパイルされ、正しい答えを返した。カーネルの再書き込みは不要だった。
- 費用の大半は探索ではない。 $N = 7$ でさえ、総時間 151 ms のうち探索は 59 ms で、残り 92 ms は HTTP 往復と JIT コンパイルである。このベンチマークは実質的に「実行系の入口の速さ」を測っている部分が多い。
- アクタ 1 体あたり約 100 μ s。 $N = 6$ で 80 μ s、 $N = 7$ で 108 μ s。生成・メッセージ 1 往復・自殺までを含む値で、二つの N で同じ桁に収まったので、スケーリングの目安として使える。
- 測れる範囲は狭い。下は雑音 (± 24 ms) に、上はアクタ表と実行予算 ($N \leq 7$) に挟まれており、実質的に $N = 6, 7$ の 2 点でしか探索費用を語れない。

4 この測定が示していないこと

- Pi 5 が Pi 4 より速いかどうかは測っていない。今回 Pi 4 と Pi 3 は応答せず、実機は Pi 5 の 1 台だけだった。
- 4 コアを使えているかも測っていない。この N-クイーンズは JIT のアクタ実行系の上で動いており、マルチコア並列は既定で無効である（別途 /smp-bench では 4 コアで 3.16 倍が出ている）。
- 分散実行の性能も測っていない。本プログラムは区間分割 (bootstrap(n, lo, hi)) で複数ノードに割れる形になっているが、今回は 1 台で lo=0, hi=N を走らせてただけである。
- /actor/send の戻り値が reply の値を運んでいないため、解の個数はボードの print 出力から読んでいる。返り値経路そのものは未検証である。

5 測定に使ったプログラム（抜粋）

現行仕様の表層構文を使っている箇所を示す。全文は次の場所にある。

~/projects/drone-hil/bench/nq_latest/nqueens_latest.aipl

```
class Node {
  var parent = nil;           // アクタ参照は nil で初期化する
  var is_root = false;       // 真偽値リテラル
  var done = false;

  method init_parent(par, sl) : unit !{mut} { // 戻り値注釈 + 効果注釈
    parent = par; parent_slot = sl;
  }

  function safe(cols, r, c) : int { ... }

  method reply(slot, v: int) : unit !{mut} { // v: int で + を整数加算に固定
    sumv = sumv + v; received = received + 1;
    if (received == expected) {
```

```

    if (is_root == true) {
        result = sumv; done = true;
        print("[NQ] n=7 solutions=" ++ result); // ++ は文字列連結
    } else { send parent.reply(parent_slot, sumv); suicide(); }
}
}
var root = new Node(); // new に引数は渡せない (send で初期化する)
send root.bootstrap(7, 0, 7);

```

6 再現方法

```

cd ~/projects/drone-hil/bench/nq_latest
python3 bench_nq.py 5          # 各 N を 5 回。results.json / summary.json が出る

```

ボードは連続したリクエストに弱いため、駆動スクリプトは各要求の間に 2.5 秒の間隔を置き、実行ごとにアクタ GC (/api/actors-gc?threshold_ms=0&dry=0) を挟んでいる。間隔を置かないと接続が落ち、ボードが死んだように見える（実際は生きている）。